Reaction Paper: The Voter Model in Opinion Dynamics

Group 36: Xinyi Cui, Leyi Yan, Ruiying Cai, Chaowei Xiao

November 4, 2024

1 Introduction

The Voter Model has provided a foundational framework in understanding opinion dynamics across social networks, capturing the ways individual choices aggregate into larger social phenomena such as consensus or polarization. Traditionally, the model simplifies interactions as pairwise and assumes immediate responses, limiting its applicability to the complexity of real-world systems where social influence is often multifaceted and delayed. Recent advancements have expanded the Voter Model, introducing models that incorporate hypergraphs [1] (to capture group-based influences), nonlinear influence [2] (to account for varying susceptibility to change), and latency [3] (to simulate delayed responses). This paper reviews three such studies that extend the classical Voter Model to more closely reflect the nuanced spread of opinions within complex networks, thereby offering a more robust tool for understanding social contagion and influence diffusion in contemporary settings.

2 Main Technical Content of the Papers

Adaptive Voter Model on Hypergraphs [1]: This paper builds on the traditional Voter Model by applying it to hypergraphs, where people are influenced by groups rather than just one-on-one interactions. It looks at two ways opinions spread: majority voting, where people follow the majority opinion in their group, and proportional voting, where people are influenced based on the proportion of opinions in the group. These approaches show that groups can either break into isolated communities or find a temporary balance where different opinions coexist. The model demonstrates how group pressure can significantly change the way opinions spread through a social network.

Nonlinear Voter Model with Multiple States [2]: The Nonlinear Voter Model (NLVM) extends the voter model to allow for multiple opinion states, with adoption probabilities that are nonlinear in nature. Depending on the nonlinearity parameter q, different situations occur: when $q > 1$, majority opinions dominate and consensus is reached quickly, whereas when $q < 1$, minority opinions are more likely to persist, creating metastable states. The paper uses pair approximations and simulations to demonstrate the model's dynamics.

Voter Model with Latency [3]: This paper introduces latency into the traditional voter model, adding a fixed delay period during which individuals cannot change their opinion after switching. The resulting system exhibits oscillatory behavior in average opinion, reflecting real-world opinion cycles such as political swings in elections. The model contrasts with the traditional voter model, which typically trends toward consensus. Instead, latency introduces memory effects that prevent immediate convergence, leading to periods of coexistence before consensus is reached

3 Relevance to Course Topics

The extended Voter Models reviewed above connect deeply with key course concepts, particularly small-world networks, threshold models of collective behavior, and cascading behavior.

3.1 Small-World Network Properties

Each model incorporates aspects that reflect **small-world network properties**, where strong local clusters coexist with a few long-range connections.

- The Adaptive Voter Model on Hypergraphs captures this by structuring influence within groups, allowing for high clustering within communities while still enabling opinion spread across different parts of the network.
- The Nonlinear Voter Model shows how localized clusters can maintain minority opinions while remaining part of a larger network where opinions can still cascade across weak ties.
- The Voter Model with Latency highlights how local clusters can experience synchronized opinion shifts, while latency introduces gradual propagation across the network, simulating the wave-like spread of opinions characteristic of small-world networks.

3.2 Threshold-Based Behavior

Each model incorporates elements of **threshold-based behavior** [4], where individuals' decisions depend on the level of influence within their social groups.

- The Adaptive Voter Model on Hypergraphs captures threshold dynamics within groups, where individuals shift their opinions based on the dominant sentiment within their hyperedge.
- The Nonlinear Voter Model uses the parameter q as an implicit threshold, allowing for different degrees of conformity or resistance to majority influence, thus enabling the persistence of minority opinions.
- The Voter Model with Latency reflects threshold effects through time delays, where individuals retain opinions for a period before reconsidering, simulating resistance to immediate opinion shifts.

3.3 Cascading Behaviors

All three models simulate **cascading behaviors** where opinion changes propagate through the network, often triggered by shifts within groups.

- In the Adaptive Voter Model on Hypergraphs, opinion cascades occur as majority opinions spread within hyperedges, causing shifts that ripple through the network.
- The Nonlinear Voter Model introduces cascading through nonlinear adoption, where changes can happen in bursts, reflecting how threshold effects can lead to sudden shifts in opinion.
- The Voter Model with Latency incorporates time delays that lead to wave-like opinion swings, showing how cascades can be staggered over time due to delayed responses.

4 Weaknesses

While each of these models offers valuable insights, there are notable limitations that could be addressed in future research.

Hypergraph-Based Voter Model: This model assumes uniform influence within hyperedges, which does not reflect the variability in influence observed in real-world networks. Additionally, reliance on synthetic simulations may limit its applicability to real-world social systems, where groups are often more complex and heterogeneous.

Nonlinear Voter Model: The model uses a fixed nonlinearity parameter for all individuals, which oversimplifies the diversity in human susceptibility to influence. In reality, individuals may have different thresholds for adopting majority views, depending on context. Furthermore, testing on complete networks limits its generalizability to real-world networks.

Voter Model with Latency: A fixed latency period for all individuals overlooks the diversity in how quickly people reconsider their opinions. In practice, response times vary, influenced by factors like attention and personal relevance. Introducing variable latency could yield more realistic insights into opinion swings.

5 Potential Future Research

5.1 Incorporating Heterogeneous Influence Levels and Individual Nonlinearity

Heterogeneous influence levels introduce a dynamic in which the impact of each individual's opinion varies based on their social capital or connectivity within the network, giving more influential agents greater influence in shaping the opinions of the group. The adaptive voter model on hypergraphs is modified to account for varying group sizes and influence hierarchies, where certain groups or individuals within a hypergraph have more persuasive power over others. And Nonlinear Voter Models could introduce individual-specific nonlinearity parameters q, reflecting how different individuals or nodes in the network are more or less susceptible to influence

5.2 Empirical Validation of Opinion Models with Social Media Data

Real-World Data Validation, focus on applying them to real-world datasets. Platforms such as Twitter, Facebook, and Reddit provide rich data on how opinions evolve over time and how influential individuals or events trigger shifts. The adaptive voter model on hypergraphs is applied to social media groups and communities to analyze how group dynamics impact the spread of political ideologies. And The Nonlinear Voter Model is tested in marketing contexts, where minority opinions often persist alongside majority trends.

5.3 Hybrid Model: A Hypergraph-Nonlinear-Latency Model for Real-Time Opinion Cascades

Creating an integrated Hypergraph-Nonlinear-Latency Voter Model that merges group-based interactions in hypergraphs with nonlinear opinion adoption dynamics and variable latency periods. This comprehensive model incorporates all three aspects, providing a more realistic simulation of opinion spread and evolution within complex, real-time social networks. By combining cascading effects and delayed responses, it enables analysis of the timing for opinion cascades to peak or decline, especially relevant in rapid-response environments like social media during political campaigns or crisis situations.

6 Promising Further Research

6.1 Proposal

The *Hybrid Hypergraph-Nonlinear-Latency Voter Model* simulates complex opinion dynamics by combining group interactions within hypergraphs, variable influence levels, and delayed opinion changes. Unlike traditional models, this approach captures multi-person influence through hyperedges, incorporates a nonlinearity parameter q to reflect varying responses to majority or minority opinions, and introduces latency periods to simulate delays in opinion shifts. This model effectively mirrors real-world opinion spread, enabling analysis of cascades, polarization, and consensus in dynamic social environments like social media and political campaigns.

6.2 Mathematical Proof

The proposed *Hybrid Hypergraph-Nonlinear-Latency Voter Model* combines three primary components: hypergraph-based group interactions, nonlinear opinion adoption influenced by a parameter q, and variable latency for opinion changes. This mathematical analysis defines the model's core processes and explores their implications for opinion dynamics.

6.2.1 Hypergraph Structure and Group Influence

In this model, opinions propagate through a hypergraph $G = (V, E)$, where:

- V is the set of nodes (individuals in the network).

- E is the set of hyperedges, where each hyperedge $e \subset V$ represents a group of individuals who can influence each other.

Each hyperedge $e \in E$ is associated with a group influence function that determines how opinions are adopted within the group. Let $X_i(t) \in \{0, 1\}$ denote the opinion of node i at time t, where 0 and 1 represent two distinct opinions.

To calculate the influence of a hyperedge on an individual's opinion, we define the influence score $S_X(e)$ for opinion X within hyperedge e as:

$$S_X(e) = \sum_{j \in e} \alpha_j \cdot \mathbf{1}(X_j = X),$$

where:

- α_j represents the influence weight of node j within the hyperedge e, accounting for heterogeneous influence levels.
- $\mathbf{1}(X_j = X)$ is an indicator function equal to 1 if $X_j = X$ and 0 otherwise.

The probability $P(X_i(t+1) = X|e)$ that a node i in hyperedge e adopts opinion X at the next time step depends on this influence score, adjusted by the nonlinearity parameter q (discussed in the next section).

6.2.2 Nonlinear Influence and Adoption Probability

The adoption probability incorporates a nonlinearity parameter q that determines how the node responds to majority versus minority opinions. Let $S_0(e)$ and $S_1(e)$ represent the influence scores for opinions 0 and 1, respectively, within hyperedge e.

The probability $P(X_i(t+1) = 1|e)$ of node i adopting opinion 1 is given by:

$$P(X_i(t+1) = 1|e) = \frac{S_1(e)^q}{S_0(e)^q + S_1(e)^q},$$

where q controls the model's sensitivity to majority or minority opinions:

- If $q > 1$, the model favors majority opinion adoption, meaning nodes are more likely to follow the dominant opinion within their group.
- If $q < 1$, the model allows for minority opinion persistence, enabling smaller opinion clusters to resist the influence of the majority.

Thus, q effectively acts as a threshold parameter, modulating susceptibility to peer influence and shaping the network's overall opinion dynamics. Higher values of q lead to quicker consensus, while lower values introduce the possibility of metastable states where multiple opinions coexist.

6.2.3 Variable Latency and Opinion Change Delay

To introduce temporal dynamics, each node is assigned a latency period L_i that determines the minimum time the node must wait before reconsidering its opinion after changing it. This latency period captures delays in opinion shifts and introduces oscillatory behavior.

Let $T_i(t)$ denote the last time node i changed its opinion. The update rule for opinion adoption then includes a latency condition:

$$X_i(t+1) = \begin{cases} X_i(t) & \text{if } t - T_i(t) < L_i, \\ \text{argmax}_X \, P(X_i(t+1) = X|e) & \text{if } t - T_i(t) \geq L_i. \end{cases}$$

Here:

- $X_i(t+1) = X_i(t)$ if the latency condition $t - T_i(t) < L_i$ is not met, meaning the node retains its current opinion.

- Once $t - T_i(t) \geq L_i$, node i becomes eligible to reconsider its opinion based on the probabilities defined by the influence and nonlinearity functions.

The latency L_i for each node can be drawn from a distribution, such as an exponential or normal distribution, to simulate variability in individuals' response times. This delay mechanism causes the spread of opinions to occur in waves or cycles, simulating real-world phenomena like periodic opinion swings in political and social contexts. See Algorithm 1 on page 9.

6.2.4 Expected Dynamics and Behavior

Combining these components, we expect the following behaviors in the Hybrid Hypergraph-Nonlinear-Latency Voter Model:

- **Consensus Formation**: With high q values and short latency, the model favors rapid consensus as majority opinions dominate and nodes have low resistance to change.
- **Minority Persistence**: Lower q values lead to metastable states where minority opinions can persist, particularly in highly clustered hyperedges with strong local influence.
- **Oscillatory Behavior**: Variable latency introduces cycles in opinion dynamics, as nodes are prevented from immediate opinion changes. This latency-driven delay results in periodic opinion swings, particularly in scenarios where the network has strong local clusters (as in small-world networks).

By analyzing these properties, we can better understand how the interplay of group influence, nonlinearity, and latency affects opinion dynamics in networks with small-world properties.

6.3 Test

6.3.1 Objectives

The purpose of this test is to validate that the Hybrid Hypergraph-Nonlinear-Latency Voter Model behaves as expected under various controlled conditions. The main goals are to confirm that:

- **Cascading behavior** occurs when majority influence is strong.
- **Minority persistence** is observed under conditions favoring opinion diversity.
- **Oscillatory patterns** emerge with adaptive latency in balanced opinion scenarios.

This test serves as a proof of concept, verifying the model's ability to capture complex social phenomena and establishing a foundation for further, larger-scale experiments.

6.3.2 Test Setup

1. **Network Configuration**:
    - Create a network of 40 nodes grouped into 6 hyperedges. Each hyperedge should include between 6-10 nodes to reflect group-based influence.
    - Assign each node an initial opinion (0 or 1) randomly to simulate an initially diverse opinion landscape.
2. **Parameter Configuration**:
    - **Nonlinearity Parameter** q:
        - Set high q values (e.g., $q = 2$) for majority influence.
        - Set low q values (e.g., $q = 0.5$) to explore scenarios favoring minority persistence.
    - **Latency Periods** L:
        - Test cases with fixed latency (e.g., $L = 3$) to observe consistent delays.

- Additional cases with variable latency (e.g., sampled from a Gaussian distribution $\mathcal{N}(3,1)$) to examine adaptive delays.

- **Consensus Threshold**:
  - Set a threshold of 80% for consensus, requiring at least 80% of nodes to hold the same opinion to declare a network consensus.

- **Simulation Duration** T:
  - Run the simulation for 100 time steps or until consensus is achieved.

6.3.3 Analysis of Test Cases

The following section provides an analysis of the observed results for each test case in the Hybrid Hypergraph-Nonlinear-Latency Voter Model. Each case explores different values for the nonlinearity parameter q and latency settings, examining how these parameters affect opinion dynamics within the network.

- **Case 1: Strong Majority Influence (High $q = 2$, Fixed Latency $L = 3$)**
  Expected Outcome: With a high q value, we anticipated strong majority influence leading to rapid consensus.
  Observed Outcome: The average opinion fluctuates initially but eventually declines towards 0 (or close to 0.2 in the displayed values), indicating that a consensus or near-consensus was reached, where most nodes adopted the same opinion.
  Explanation: The high q promotes majority opinion adoption, so the nodes tend to align with the most popular opinion in their groups. The fixed latency introduces some delay, but overall, the model shows rapid convergence to a single dominant opinion.

- **Case 2: Minority Persistence (Low $q = 0.5$, Fixed Latency $L = 3$)**
  Expected Outcome: Lower q should allow minority opinions to persist, leading to a metastable state where both opinions coexist.
  Observed Outcome: The average opinion oscillates significantly without settling, indicating that both opinions are present in the network over time.
  Explanation: The low q value reduces the pressure to adopt the majority opinion, allowing minority views to survive. The fixed latency adds delay, leading to persistent fluctuations, as nodes change opinions periodically rather than achieving a steady consensus.

- **Case 3: Oscillatory Dynamics (Moderate $q = 1.5$, Gaussian-Distributed Latency $L \sim \mathcal{N}(3,1)$)**
  Expected Outcome: With moderate q and Gaussian-distributed latency, we anticipated oscillatory dynamics and delayed opinion shifts.
  Observed Outcome: The average opinion initially oscillates with a slight upward trend but eventually declines, suggesting temporary opinion swings before a final shift towards a single dominant opinion.
  Explanation: The moderate q and Gaussian-distributed latency cause nodes to periodically switch opinions. However, as some nodes experience shorter latency periods than others, clusters of opinion change propagate throughout the network, ultimately favoring one opinion after several oscillations.

- **Case 4: Mixed Influence and Latency (Moderate $q = 1$, Gaussian-Distributed Latency $L \sim \mathcal{N}(3,2)$)**
  Expected Outcome: Partial consensus with minor oscillations due to the mix of influence and latency settings.
  Observed Outcome: There are noticeable oscillations in the average opinion, and no clear convergence is observed within the given time frame.
  Explanation: The moderate q value and Gaussian-distributed latency create a balanced dynamic where nodes do not strictly follow the majority or minority opinion, resulting in sustained opinion diversity. This scenario produces oscillations, as the network does not reach full consensus within the time frame.

6.3.4 Summary

- **High** q (Case 1) favors rapid consensus, as expected, while **low** q (Case 2) allows for significant opinion diversity and ongoing fluctuations.
- **Moderate** q **with Gaussian-distributed latency** (Case 3) leads to temporary oscillations, which eventually converge towards a dominant opinion.
- **Moderate** q **with Gaussian-distributed mixed latency** (Case 4) produces sustained oscillations, with the network maintaining diverse opinions.

These results demonstrate the model's sensitivity to the nonlinearity parameter q and latency distribution, providing insight into how opinion dynamics may unfold in networks with varying degrees of influence and delay. See code in Appendix.

References

[1] A. Golovin, J. Mölter, and C. Kuehn, *Polyadic opinion formation: The adaptive voter model on a hypergraph*, 2023. arXiv: 2308.03640 [physics.soc-ph]. [Online]. Available: https://arxiv.org/abs/2308.03640.

[2] L. S. Ramirez, F. Vazquez, M. S. Miguel, and T. Galla, *Ordering dynamics of nonlinear voter models*, 2023. arXiv: 2310.20548 [physics.soc-ph]. [Online]. Available: https://arxiv.org/abs/2310.20548.

[3] G. Palermo, A. Mancini, A. Desiderio, R. Di Clemente, and G. Cimini, "Spontaneous opinion swings in the voter model with latency," *Physical Review E*, vol. 110, no. 2, Aug. 2024, ISSN: 2470-0053. DOI: 10.1103/physreve.110.024313. [Online]. Available: http://dx.doi.org/10.1103/PhysRevE.110.024313.

[4] M. Granovetter, "Threshold models of collective behavior," *American Journal of Sociology*, vol. 83, no. 6, pp. 1420–1443, 1978.

Appendix

Algorithm 1 Hybrid Hypergraph-Nonlinear-Latency Voter Model

Input:
Nodes V: Set of individuals in the network
Hyperedges E: Set of groups (hyperedges) in the network, each containing multiple nodes
Opinions X: Set of possible opinions, e.g., $X = \{0, 1\}$
Influence weights α: Influence weight for each node in a hyperedge
Nonlinearity parameter q: Controls sensitivity to majority/minority opinion
Latency periods L: Latency period L_i for each node i
Time steps T: Total simulation time steps
consensus_threshold: Proportion of nodes with the same opinion required for consensus

Initialize:
Assign each node $i \in V$ an initial opinion $X_i(0) \in X$
Assign each node i a latency period L_i
Initialize the time of the last opinion change for each node $T_i \leftarrow 0$

Simulation:
for $t = 1$ to T **do**
  for each hyperedge e in E **do**
    for each node i in e **do**
      if $t - T_i \geq L_i$ **then**
        $S_0 \leftarrow 0$
        $S_1 \leftarrow 0$
        for each node j in e **do**
          if $X_j = 0$ **then**
            $S_0 \leftarrow S_0 + \alpha_j$
          else if $X_j = 1$ **then**
            $S_1 \leftarrow S_1 + \alpha_j$
          end if
        end for
        $P_0 \leftarrow \frac{S_0^q}{S_0^q + S_1^q}$
        $P_1 \leftarrow \frac{S_1^q}{S_0^q + S_1^q}$
        if random() $< P_1$ **then**
          $new_opinion \leftarrow 1$
        else
          $new_opinion \leftarrow 0$
        end if
        if $X_i \neq new_opinion$ **then**
          $X_i \leftarrow new_opinion$
          $T_i \leftarrow t$
        end if
      end if
    end for
  end for
  if proportion of same opinion nodes $\geq$ *consensus_threshold* **then**
    Consensus reached, exit loop
  end if
end for

Output:
Final opinion distribution across nodes
Time to reach consensus (if achieved within T)
Proportion of nodes holding each opinion at each time step
Number of opinion oscillations observed over time (if applicable)

```python
import numpy as np
import random
import matplotlib.pyplot as plt

# Set the random seed for reproducibility
np.random.seed(42)

class VoterModel:
    def __init__(self, num_nodes, hyperedges, q, latency, consensus_threshold):
        self.num_nodes = num_nodes
        self.hyperedges = hyperedges
        self.q = q
        self.latency = latency # Either fixed or a distribution
        self.consensus_threshold = consensus_threshold
        self.opinions = np.random.choice([0, 1], size=num_nodes)
        self.last_change_time = np.zeros(num_nodes, dtype=int)
        self.time_step = 0
        self.history = []

    def influence_score(self, opinion, edge):
        return sum(1 for node in edge if self.opinions[node] == opinion)

    def update_opinion(self):
        new_opinions = self.opinions.copy()
        for edge in self.hyperedges:
            s0 = self.influence_score(0, edge) ** self.q
            s1 = self.influence_score(1, edge) ** self.q
            for node in edge:
                if self.time_step - self.last_change_time[node] >= self.latency[node]:
                    p1 = s1 / (s0 + s1) if s0 + s1 > 0 else 0.5
                    new_opinions[node] = 1 if random.random() < p1 else 0
                    if new_opinions[node] != self.opinions[node]:
                        self.last_change_time[node] = self.time_step
        self.opinions = new_opinions

    def check_consensus(self):
        majority_opinion = np.mean(self.opinions)
        return majority_opinion >= self.consensus_threshold or majority_opinion <= 1 -
            self.consensus_threshold

    def run(self, max_steps=100):
        for self.time_step in range(1, max_steps + 1):
            self.history.append(np.mean(self.opinions))
            if self.check_consensus():
                print(f"Consensus reached at step {self.time_step}")
                break
            self.update_opinion()
        return self.history

    def plot_history(self):
        plt.plot(self.history, label=f'q={self.q}, latency={self.latency}')
        plt.xlabel("Time Step")
        plt.ylabel("Average Opinion")
        plt.title("Opinion Dynamics Over Time")
        plt.legend()
        plt.show()

# Parameters for Test Cases
```

```
num_nodes = 40
hyperedges = [random.sample(range(num_nodes), k=random.randint(6, 10)) for _ in range(6)]
consensus_threshold = 0.8
max_steps = 100

# Test Case 1: Strong Majority Influence
q_high = 2
latency_fixed = np.full(num_nodes, 3)
model1 = VoterModel(num_nodes, hyperedges, q=q_high, latency=latency_fixed,
    consensus_threshold=consensus_threshold)
history1 = model1.run(max_steps=max_steps)
model1.plot_history()

# Test Case 2: Minority Persistence
q_low = 0.5
latency_fixed = np.full(num_nodes, 3)
model2 = VoterModel(num_nodes, hyperedges, q=q_low, latency=latency_fixed,
    consensus_threshold=consensus_threshold)
history2 = model2.run(max_steps=max_steps)
model2.plot_history()

# latency
latency_variable = np.random.randint(1, 6, size=num_nodes)
latency_mixed = np.random.randint(2, 4, size=num_nodes)
latency_gassian1 = np.clip(np.random.normal(3,1,size=num_nodes), 1, None).astype(int)
latency_gassian2 = np.clip(np.random.normal(3,2,size=num_nodes), 1, None).astype(int)

# Test Case 3: Oscillatory Dynamics
q_moderate = 1.5
model3 = VoterModel(num_nodes, hyperedges, q=q_moderate, latency=latency_gassian1,
    consensus_threshold=consensus_threshold)
history3 = model3.run(max_steps=max_steps)
model3.plot_history()

# Test Case 4: Mixed Influence and Latency
q_moderate = 1
model4 = VoterModel(num_nodes, hyperedges, q=q_moderate, latency=latency_gassian1,
    consensus_threshold=consensus_threshold)
history4 = model4.run(max_steps=max_steps)
model4.plot_history()
```